

CIBERTOOLBOX

Relatório de Evolução do Projeto

Da versão 1 ao estágio atual de desenvolvimento

Projeto academico de conclusao de curso - TCC

Desenvolvimento em Python, CustomTkinter e arquitetura baseada em MVC

2026

Nota sobre o escopo deste documento

Este relatório foi elaborado a partir da análise direta dos arquivos fornecidos da versão 1 do CiberToolBox e do histórico de desenvolvimento da versão atual discutido ao longo da orientação do projeto. A versão 1 foi inspecionada nos arquivos Database.py, LoginWindow.py, main.py, no banco SQLite e na estrutura de pastas. As informações sobre a versão atual refletem as decisões de arquitetura, interface, navegação e planejamento já implementadas ou definidas até este momento.

O documento tem finalidade de acompanhamento técnico e acadêmico. Ele pode servir como base para a seção de desenvolvimento do TCC, para apresentações de orientação e para a preparação da defesa, mas ainda deverá ser ajustado às normas da instituição e complementado com referências bibliográficas, resultados de testes e dados da pesquisa de campo.

Sumário

1. Introdução
 2. Objetivos do relatório
 3. Visão geral do projeto
 4. Análise da versão 1
 5. Limitações identificadas
 6. Transição para a versão atual
 7. Arquitetura e separação de responsabilidades
 8. Evolução da interface e identidade visual
 9. Dashboard de ferramentas
 10. Ferramentas, serviços e configurações
 11. Segurança e uso ético
 12. Metodologia de desenvolvimento
 13. Comparação entre as versões
 14. Situação atual
 15. Plano de conclusão
 16. Riscos e cuidados
 17. Considerações finais
- Apendice A - Roteiro de apresentação oral
- Apendice B - Glossário técnico

1. Introdução

O CiberToolBox é uma aplicação desktop idealizada para centralizar ferramentas de apoio à cibersegurança, diagnóstico de redes, verificação de integridade e orientação a usuários iniciantes. A proposta procura reduzir a dispersão causada pelo uso de diferentes programas, comandos de terminal e interfaces para tarefas que podem fazer parte de um mesmo fluxo de trabalho.

O projeto nasceu como um protótipo funcional. A primeira versão reuniu autenticação com SQLite, uma janela principal e um terminal interno que permitia executar comandos associados ao Sherlock e ao Nmap. Essa versão cumpriu o papel de validar a ideia inicial, mas também revelou problemas de organização, segurança, manutenção e expansão.

A versão atual representa uma reformulação do projeto. O sistema passou a adotar uma estrutura inspirada no padrão MVC, uma identidade visual própria, um mascote como elemento central de navegação e uma dashboard dinâmica de ferramentas. A proposta deixou de ser apenas um terminal gráfico e passou a ser uma plataforma organizada, educativa e progressivamente modular.

2. Objetivos do relatório

O objetivo geral deste relatório é registrar a evolução do CiberToolBox desde sua primeira versão até o estágio atual, demonstrando as decisões técnicas, visuais e metodológicas adotadas.

Como objetivos específicos, o documento busca:

- descrever a estrutura e o funcionamento da versão 1;
- identificar pontos positivos e limitações do protótipo inicial;
- explicar a reorganização arquitetural da versão atual;
- documentar a evolução das telas de login, tela principal e dashboard;
- definir a separação entre View, Controller, Model e classes de serviço;
- registrar as funcionalidades planejadas;
- apresentar uma sequência realista para a conclusão do projeto.

3. Visão geral do projeto

O público-alvo principal é formado por estudantes, técnicos iniciantes e usuários que estejam aprendendo conceitos de redes e segurança da informação. Por esse motivo, o projeto não deve apenas executar comandos: ele deve explicar a finalidade de cada recurso, orientar sobre o uso correto e apresentar resultados em linguagem acessível.

A ideia central pode ser resumida no seguinte fluxo:

```
Login
-> Tela principal com mascote
  -> Ferramentas
  -> Serviços
  -> Configurações
```

O diferencial pretendido não é competir diretamente com suites profissionais de segurança, mas organizar funções essenciais em uma interface única, com identidade própria, foco educacional e preocupação com o uso ético.

4. Analise da versão 1

4.1 Estrutura de arquivos

A estrutura encontrada na versão 1 foi a seguinte:

```
CiberToolBox v1/
|- Bd/
|  `-- LoginDataBase.db
|- Ferramentas/
|   |-- ferramentas.txt
|   `-- sherlock readme.txt
|- Anotacoes.txt
|- Database.py
|- LoginWindow.py
|- main.py
`-- requirement.txt
```

A organização corresponde a uma fase de prototipação. Os arquivos principais estavam na raiz, sem separação formal entre interface, banco de dados, controle e lógica das ferramentas.

4.2 Login e cadastro

O arquivo LoginWindow.py implementava as telas de login e cadastro utilizando o Tkinter tradicional. A tela de login possuía campos de e-mail e senha, botão de entrada e acesso ao cadastro. A tela de cadastro permitia inserir nome, e-mail ou usuário e senha.

A verificação de credenciais era realizada pela classe Funcs, presente em Database.py. O banco SQLite possuía a tabela clientes com código, nome, e-mail e senha.

```
SELECT * FROM clientes
WHERE email_cliente = ?
AND senha_cliente = ?
```

O uso de parâmetros representados por sinais de interrogação foi uma escolha correta, pois evita concatenar diretamente os dados fornecidos pelo usuário na consulta SQL. Isso reduz o risco de injeção de SQL nessa operação.

Entretanto, as senhas eram gravadas e comparadas em texto puro. Essa é uma limitação relevante, pois uma pessoa com acesso ao banco poderia visualizar as credenciais. Em uma evolução posterior, as senhas devem ser processadas com hash seguro e salt, utilizando biblioteca apropriada.

4.3 Tela principal e terminal interno

O arquivo main.py criava a classe MainWindows com uma barra lateral e uma área de conteúdo. Os botões Sherlock e Nmap abriam uma simulação de terminal dentro da aplicação.

A entrada digitada era capturada pelo evento Return, separada do prompt visual e enviada para execução em uma thread. A interface mantinha uma lista de comandos permitidos para cada ferramenta.

```
permitidos = ["sherlock", "ls", "cd", "cls", "clear",
              "dir", "python3", "help", "man"]
```

A verificação utilizava o primeiro termo do comando como base. Se o termo não estivesse na lista, o sistema exibía uma mensagem de acesso negado. Essa estratégia é conhecida como whitelist ou lista branca.

4.4 Uso de threads

O processamento dos comandos era iniciado com `threading.Thread` e a opcao `daemon=True`. Essa decisao evitava que comandos demorados bloqueassem imediatamente a interface principal. O conceito e adequado: tarefas longas nao devem ser executadas diretamente na thread grafica.

Apesar disso, a thread modificava diretamente o `CTkTextbox`. Em Tkinter e CustomTkinter, atualizacoes de widgets devem ocorrer preferencialmente na thread principal. A versao futura deve executar a operacao em segundo plano e devolver o resultado para a interface por meio de `after`, fila ou callback seguro.

4.5 Execucao pelo PowerShell

A execucao era feita com `subprocess.run`, chamando o PowerShell e passando o comando completo. Isso tornou o prototipo dependente do Windows e aumentou a superficie de risco. Mesmo com whitelist, comandos com parametros, mudanca de diretorio e chamadas a interpretadores podem produzir comportamentos inesperados.

A evolucao proposta substitui o terminal generico por telas especificas para cada ferramenta. Assim, o usuario fornece apenas os parametros esperados, e a aplicacao monta internamente uma chamada controlada.

4.6 Pontos positivos da versao 1

- validacao inicial da proposta de centralizar ferramentas;
- uso de SQLite para persistencia;
- consultas SQL parametrizadas;
- primeira tentativa de controle por whitelist;
- uso de thread para evitar congelamento durante tarefas demoradas;
- interface grafica capaz de abrir ferramentas diferentes;
- base pratica para identificar problemas e orientar a refatoracao.

5. Limitacoes identificadas

- interface e logica de execucao concentradas no mesmo arquivo;
- ausencia de separacao clara de responsabilidades;
- criacao de uma nova instancia CTK e um novo mainloop apos o login;
- senhas armazenadas em texto puro;
- dependencia direta do PowerShell;
- execucao de comando textual amplo em vez de parametros controlados;
- atualizacao de widget a partir de thread secundaria;
- estrutura de pastas pouco escalavel;
- presenca de arquivos `__pycache__` no pacote do projeto;
- arquivo de dependencias com nomenclatura incorreta e documentacao limitada;
- identidade visual ainda nao definida;
- dificuldade para adicionar novas ferramentas sem aumentar o acoplamento.

Essas limitacoes nao anulam o valor do prototipo. Elas mostram que a versao 1 cumpriu seu papel como prova de conceito e forneceu informacoes importantes para a versao seguinte.

6. Transicao para a versao atual

A reformulacao teve como prioridade organizar o codigo antes de ampliar o numero de recursos. A estrutura passou a ser distribuida em pastas para Models, Views, Controller e Servicos. Essa divisao busca aproximar o projeto do padrao MVC sem tornar o codigo excessivamente complexo para o nivel atual de desenvolvimento.

```
src/
|- Controller/
|- Models/
|- Views/
|- Servicos/
|- Ferramentas/
```

A transicao tambem envolveu a substituicao do terminal generico por um fluxo de telas. Cada ferramenta devera ter uma interface propria, validacao especifica e uma classe responsavel por sua operacao tecnica.

7. Arquitetura e separacao de responsabilidades

7.1 View

A View e responsavel por apresentar informacoes e receber interacoes. Ela cria campos, botoes, labels, paineis e mensagens. Exemplos atuais sao login_view.py, main_window.py e tools_view.py.

A View pode ler um campo e exibir um resultado, mas nao deve executar diretamente subprocess.run, socket.connect ou hashlib. Essa regra reduz o acoplamento entre interface e logica.

7.2 Controller

O Controller atua como coordenador. Ele recebe a solicitacao da View, realiza validacoes de fluxo, chama a classe correta e devolve um resultado padronizado. Ele nao deve concentrar toda a logica da ferramenta.

```
View recebe o host
-> Controller valida o pedido
   -> PingTool executa a operacao
       -> ToolResult retorna o resultado
           -> View apresenta ao usuario
```

A regra pratica adotada e: o Controller decide quem deve trabalhar; a classe da ferramenta realiza o trabalho.

7.3 Model

O Model representa dados e estados do dominio. Exemplos planejados incluem Usuario, ToolResult e HistoryEntry. Um ToolResult pode padronizar sucesso, mensagem e saida, evitando que cada ferramenta devolva formatos diferentes.

```
@dataclass
class ToolResult:
    sucesso: bool
    mensagem: str
    saida: str = ""
```

7.4 Classes de serviço e ferramentas

As classes presentes em Serviços/Ferramentas executam a operação técnica. E nessa camada que podem aparecer subprocess, socket, hashlib e outras bibliotecas relacionadas ao funcionamento real da ferramenta.

Essa separação facilita testes unitários, pois a lógica pode ser testada sem abrir janelas. Também permite trocar a interface sem reescrever a operação técnica.

8. Evolução da interface e identidade visual

8.1 Tela de login

A tela de login foi reformulada com CustomTkinter, paleta escura e dimensões mais adequadas. O fluxo foi ajustado para manter um único mainloop. A janela de login funciona como raiz, e a tela principal é aberta como CTKToplevel, evitando a criação de ciclos de eventos concorrentes.

Também foram orientadas correções como não imprimir senhas, não transformar senhas automaticamente em maiúsculas e evitar `time.sleep` na interface.

8.2 Mascote e identidade visual

A versão atual adotou um mascote original: um ornitorrinco cibernético em tons vermelho e laranja, com chapéu branco e olhos escuros. O chapéu branco associa visualmente o projeto ao conceito de white hat, reforçando o foco defensivo e autorizado.

O mascote não é apenas decorativo. Ele funciona como botão central da navegação e ajuda a tornar o sistema memorável para o público iniciante.

8.3 Tela principal

O arquivo `main_window.py` organiza a tela principal em barra superior, cabeçalho, área interativa e barra inferior. Ao clicar no mascote, surgem os botões Ferramentas, Serviços e Configurações.

A animação usa o método `after`. Em vez de bloquear a interface, o programa agenda pequenos passos de movimento. Uma curva de suavização faz o movimento começar e terminar de forma gradual.

```
progresso_suave = progresso * progresso * (3 - 2 * progresso)
```

As imagens são processadas pela biblioteca Pillow. O código realiza recorte central, redimensionamento, máscara circular, ajuste de brilho, zoom e deslocamento interno. Isso permite aumentar ou mover apenas o conteúdo visual sem alterar o tamanho ou a posição do botão.

9. Dashboard de ferramentas

A dashboard de ferramentas foi criada como um catálogo dinâmico. Ela apresenta menu lateral, categorias, pesquisa, cartões, painel de detalhes, status de disponibilidade e botão de execução.

As ferramentas ficam organizadas em uma lista de dicionários. Cada dicionário possui nome, ícone, categoria, descrição, explicação, status, cor e ação. A interface percorre essa lista e cria os cartões automaticamente.

```
{
  "nome": "Ping",
```

```

"categoria": "Rede",
"descricao": "Verifica conectividade...",
"status": "Disponível",
"acao": self.executar_ping,
}

```

Essa abordagem é orientada a dados. Para adicionar uma nova ferramenta, o desenvolvedor inclui um novo dicionário, sem precisar duplicar toda a estrutura visual.

9.1 Categorias

- Todas - visão geral do catálogo;
- Rede - conectividade e diagnóstico;
- Integridade - hash e verificação de arquivos;
- Credenciais - avaliação de senhas;
- Sistema - informações e diagnósticos locais.

9.2 Pesquisa dinâmica

O campo de pesquisa reage ao evento KeyRelease. A cada tecla liberada, a lista é filtrada pelo nome e pela descrição. A conversão para letras minúsculas permite pesquisa sem diferença entre maiúsculas e minúsculas.

9.3 Painel de detalhes

Ao selecionar um cartão, o painel lateral apresenta explicação, categoria, status e opção de abertura. Ferramentas ainda não implementadas permanecem com o botão desabilitado. Isso evita prometer uma funcionalidade inexistente e comunica claramente o estado de desenvolvimento.

10. Ferramentas, serviços e configurações

10.1 Ferramentas técnicas

- Ping;
- Consulta DNS;
- Hash de texto;
- Hash de arquivo e comparação;
- Analisador de força de senha;
- Scanner básico e controlado de portas;
- Informações do sistema.

O primeiro ciclo funcional recomendado é o Ping completo, pois ele cria um modelo que poderá ser repetido nas demais ferramentas: tela própria, Controller, classe de serviço, resultado padronizado, tratamento de erro e execução sem travar a interface.

10.2 Serviços

A área Serviços não deve duplicar as ferramentas. Ela concentra recursos de apoio e organização:

- Histórico de atividades;
- Geração de relatórios;
- Central de ajuda e tutoriais;

- Jogos extras como recurso secundario;
- Sobre o projeto;
- Verificacao de atualizacoes em uma etapa futura.

O historico nao deve salvar senhas, credenciais ou dados sensiveis desnecessarios. O relatorio deve registrar apenas informacoes relevantes e autorizadas.

10.3 Configuracoes

- Tema e aparencia;
- Cor de destaque;
- Perfil do usuario;
- Sons;
- Informacoes da versao;
- Licenca;
- Preferencias persistidas em JSON ou SQLite.

11. Seguranca e uso etico

O projeto foi direcionado para uso defensivo e educacional. Ferramentas de rede devem apresentar avisos de autorizacao, e a demonstracao para a banca deve utilizar localhost, laboratorio proprio ou ambiente expressamente autorizado.

A versao final deve observar os seguintes principios:

- validar todas as entradas;
- limitar intervalos e timeouts;
- nao executar comandos fornecidos livremente pelo usuario;
- nao armazenar senhas analisadas;
- nao registrar credenciais em logs;
- usar hash seguro para autenticacao;
- tratar excecoes sem revelar informacoes internas desnecessarias;
- explicar as limitacoes de cada resultado.

No caso do Ping, por exemplo, a ausencia de resposta nao comprova que um equipamento esteja desligado, pois pacotes ICMP podem ser bloqueados. A interface educativa deve explicar esse tipo de limite.

12. Metodologia de desenvolvimento

12.1 Desenvolvimento incremental

O projeto esta sendo desenvolvido por incrementos. Primeiro foi criado um prototipo. Em seguida, os problemas foram analisados, a arquitetura foi reorganizada e as telas principais foram consolidadas. As ferramentas serao implementadas uma a uma.

```
Planejamento
-> Prototipo
  -> Avaliacao
    -> Refatoracao
      -> Nova funcionalidade
        -> Testes
```

Essa abordagem é adequada para um desenvolvedor junior porque reduz o número de problemas tratados ao mesmo tempo e permite validar cada etapa antes da seguinte.

12.2 Programação orientada a objetos

As telas e serviços são representados por classes. Dados e comportamentos relacionados ficam agrupados. Exemplos são ClientView, ToolsView e futuramente PingTool.

12.3 Modularização e reutilização

A interface foi dividida em métodos menores, como `criar_barra_superior`, `criar_mascote`, `criar_card` e `mostrar_ferramentas`. Métodos reutilizáveis reduzem repetição e facilitam manutenção.

12.4 Tratamento de exceções

Blocos `try` e `except` são usados para lidar com diferenças de sistema operacional, falhas de arquivo e erros de execução. O objetivo não é esconder erros, mas impedir que uma falha previsível encerre a aplicação de forma descontrolada.

12.5 Interface não bloqueante

Animações usam `after`. Operações longas deverão usar `thread` ou `executor`, mas a atualização dos widgets deve retornar para a thread principal. Essa metodologia mantém a janela responsiva.

13. Comparação entre as versões

Aspecto	Versão 1	Versão atual
Interface	Tkinter simples e terminal interno	CustomTkinter, identidade própria e navegação animada
Arquitetura	Código concentrado	Separação inicial em Views, Controller, Models e Serviços
Login	SQLite com senha em texto puro	Fluxo reorganizado; segurança de senha ainda deve evoluir
Navegação	Menu lateral com Sherlock e Nmap	Mascote central, Ferramentas, Serviços e Configurações
Ferramentas	Comandos de PowerShell	Telas específicas e catálogo modular planejado
Expansão	Exige alterações manuais extensas	Cartões gerados por dados e serviços separados
Usabilidade	Pouca explicação	Descrições, status e painel educativo
Ética	Controle por whitelist	Avisos de autorização e escopo defensivo explícito
Portabilidade	Forte dependência do Windows	Ainda desktop; operações devem tratar diferenças de sistema
Testes	Não estruturados	Testes unitários e funcionais planejados

14. Situacao atual

No estagio atual, o projeto possui uma base visual e arquitetural consistente para continuar o desenvolvimento. Os componentes ja consolidados ou encaminhados incluem:

- tela de login reformulada;
- tela principal com mascote;
- menu animado;
- identidade visual;
- processamento e ajuste de imagens;
- dashboard de ferramentas;
- categorias e pesquisa;
- painel de detalhes;
- status de disponibilidade;
- estrutura inicial inspirada em MVC.

Ainda precisam ser concluidos:

- Ping funcional;
- Consulta DNS;
- Hash de texto e arquivo;
- Analisador de senha;
- Scanner controlado de portas;
- Historico;
- Relatorios;
- Central de ajuda;
- Configuracoes persistentes;
- autenticao segura;
- testes automatizados e manuais;
- pesquisa de campo;
- documentacao final e preparacao da defesa.

15. Plano de conclusao

1. Conectar a dashboard a uma tela Ping real.
2. Criar PingTool, ToolsController e ToolResult.
3. Executar o Ping em segundo plano e exibir o resultado com segurança.
4. Usar o mesmo modelo para DNS, Hash e Analise de Senha.
5. Implementar Scanner de Portas com limite de intervalo, timeout e aviso etico.
6. Criar historico sem registrar informacoes sensiveis.
7. Criar relatorio a partir de registros selecionados.
8. Criar Central de Ajuda com explicacoes para iniciantes.
9. Implementar configuracoes e persistencia.
10. Revisar login e aplicar armazenamento seguro de senha.
11. Criar testes unitarios para regras sem interface.
12. Executar testes funcionais do fluxo completo.
13. Aplicar pesquisa de campo com tarefas e questionario.
14. Analisar resultados e finalizar a monografia.
15. Preparar slides, roteiro de demonstracao e plano alternativo em video.

16. Riscos e cuidados

- Continuar alterando o visual por muito tempo e atrasar o núcleo funcional;
- implementar muitas ferramentas incompletas em vez de poucas bem finalizadas;
- misturar lógica técnica dentro das Views;
- criar um Controller muito grande;
- bloquear a interface durante operações de rede;
- usar comandos de shell com entrada livre;
- armazenar dados sensíveis no histórico;
- demonstrar scanner em ambiente sem autorização;
- depender de conexão externa durante a apresentação;
- não reconhecer limitações técnicas e metodológicas.

A prioridade deve ser um fluxo completo e confiável. Para a banca, uma ferramenta Ping bem implementada, explicada e testada vale mais do que várias funções apenas simuladas.

17. Considerações finais

A evolução do CiberToolBox demonstra uma mudança relevante entre o protótipo inicial e a versão atual. A versão 1 validou autenticação, persistência, interface e execução de comandos. Ao mesmo tempo, revelou problemas que seriam difíceis de sustentar em um projeto maior.

A versão atual apresenta uma proposta mais coerente com um TCC de desenvolvimento de software. O projeto passou a considerar arquitetura, separação de responsabilidades, experiência do usuário, identidade visual, uso ético, manutenibilidade e validação com usuários.

O próximo marco não deve ser uma nova alteração estética, mas a entrega do primeiro ciclo funcional completo: dashboard, PingView, Controller, PingTool, resultado e tratamento de erros. A partir desse modelo, as demais ferramentas poderão ser adicionadas de forma progressiva e consistente.

O CiberToolBox ainda está em desenvolvimento, mas já possui uma base capaz de sustentar a implementação do núcleo técnico e a posterior avaliação acadêmica de sua utilidade para técnicos iniciantes.

Apêndice A - Roteiro de apresentação oral

Uma explicação curta e segura para apresentar a evolução do projeto pode ser organizada da seguinte maneira:

"O CiberToolBox começou como um protótipo com login em SQLite e um terminal interno para Sherlock e Nmap. A primeira versão permitiu validar a ideia, mas misturava interface e lógica, dependia do PowerShell e armazenava senhas em texto puro. Na versão atual, reorganizei o projeto com uma estrutura baseada em MVC, criei uma identidade visual própria, um mascote como elemento de navegação e uma dashboard dinâmica. Agora o sistema entra na fase de implementação das ferramentas, com separação entre View, Controller e classes de serviço. O objetivo final é oferecer uma plataforma centralizada, educativa e acessível para técnicos iniciantes."

Apêndice B - Glossário técnico

Termo	Explicação
MVC	Padrão de organização que separa dados, interface e controle do fluxo.
View	Camada responsável pela interface e pela

	apresentacao de informacoes.
Controller	Camada que coordena a solicitacao entre a interface e a logica.
Model	Representacao dos dados e estados usados pelo sistema.
Service	Classe que executa uma operacao tecnica ou regra especifica.
Whitelist	Lista de itens previamente permitidos.
Thread	Fluxo de execucao separado usado para tarefas que poderiam bloquear a interface.
mainloop	Ciclo principal que processa eventos de uma interface Tkinter.
CTkToplevel	Janela secundaria ligada a uma aplicacao CustomTkinter existente.
RGBA	Formato de cor com canais vermelho, verde, azul e transparencia.
Hash	Resumo matematico de tamanho fixo usado, entre outras finalidades, para verificacao de integridade.
Timeout	Tempo limite para uma operacao ser encerrada quando nao responde.
Acoplamento	Nivel de dependencia entre partes do codigo.
Refatoracao	Reorganizacao do codigo sem alterar necessariamente o objetivo funcional.
MVP	Versao minima do produto que entrega o fluxo principal de forma funcional.